■ AI Project Mentor — Fix Summary Report

Date & Time: 2026-11-03 17:22:46
Session Goal / Focus: ■■■■■■■■■■■■ ■■■■■■■■■■■■, ■■■■■■ ■■■■■■■■■■■■ ■■■■■ ■■■■■■■ ■■■■ ■■■■■■

Total Fixes Applied: 5

---

1. Add Code Generation Quality Standards

Target File:	src/skills/SKILL.md
Category:	improvement
Why / ROI:	Eliminates incomplete code fixes, placeholder comments, and syntax regressions generated by the agent.

Before (Original Code):

```
# Then Make it Right
## Fix Root Causes
* Reproduce it. Fix the cause. A guard that only hides the crash is not a fix.

## Sequence work into verifiable units
* Finish one small unit and check it. Then start the next.
```

After (Applied Fix):

```
# Then Make it Right
## Fix Root Causes
* Reproduce it. Fix the cause. A guard that only hides the crash is not a fix.

## Sequence work into verifiable units
* Finish one small unit and check it. Then start the next.

# Code Generation Quality Rules
* Never output placeholders, ellipses (...), or comments like '# rest of code'. Write complete, production-ready code.
* Preserve the exact indentation style (spaces/tabs) of the target file.
* Always include necessary imports and type annotations.
* Practice defensive coding: handle None, network/IO failures, and boundary edge cases.
* Keep changes minimal, robust, and verifiable.
```

2. Enforce Strict Code Quality in Prompt

Target File:	src/agent.py
Category:	improvement
Why / ROI:	Significantly boosts the accuracy of ready_code_fix snippets and search_block matching rate.

Before (Original Code):

```
"STRICT SEARCH BLOCK RULE: 'search_block' MUST include full lines from start to end (including function definitions or full conditionals if modifying logic). Never cut off a search_block mid-statement."
"Strictly apply your engineering skill principles (First Make it Work, Laziness Protocol, "
"Subtract Before You Add, Boundary Discipline, Fix Root Causes) when proposing tasks.\n"
f"{focus_instruction}\n"
"Analyze the provided project source code:\n"
"1. Summarize the core purpose of the project (project_summary).\n"
"2. Break findings down into bite-sized micro-tasks taking 10-20 minutes each.\n"
"3. For EVERY task, provide 'search_block' with the EXACT, VERBATIM original code to replace (must match characters byte-for-byte without ellipses or omissions), and 'ready_code_fix' with the exact replacement.\n"
"4. Keep search_block minimal (3-15 lines) around the exact lines to modify so automated string substitution succeeds reliably.\n"
"5. ATOMIC DEPENDENCY RULE: If modifying 'file_path' requires adjustments in OTHER files (such as imports, callers, or configs), include them in 'dependent_patches' with exact search_block and ready_code_fix so they can be auto-applied in the same step.\n\n"
"Use line numbers from context to locate code, but DO NOT include line number prefixes inside search_block or ready_code_fix snippets.\n\n"
f"PROJECT SOURCE CO
```

After (Applied Fix):

```
"STRICT SEARCH BLOCK RULE: 'search_block' MUST include full lines from start to end (including function definitions or full conditionals if modifying logic). Never cut off a search_block mid-statement.\n"
"STRICT READY CODE FIX RULE: 'ready_code_fix' MUST contain 100% executable, complete replacement code. NEVER use placeholders, ellipses (...), or comments like '# rest of function remains the same'.\n"
"Strictly apply your engineering skill principles (First Make it Work, Laziness Protocol, "
"Subtract Before You Add, Boundary Discipline, Fix Root Causes) when proposing tasks.\n"
f"{focus_instruction}\n"
"Analyze the provided project source code:\n"
"1. Summarize the core purpose of the project (project_summary).\n"
"2. Break findings down into bite-sized micro-tasks taking 10-20 minutes each.\n"
"3. For EVERY task, provide 'search_block' with the EXACT, VERBATIM original code to replace (must match characters byte-for-byte without ellipses or omissions), and 'ready_code_fix' with the exact replacement.\n"
"4. Keep search_block minimal (3-15 lines) around the exact lines to modify so automated string substitution succeeds reliably.\n"
"5. ATOMIC DEPENDENCY RULE: If modifying 'file_path' requires adjustments in OTHER files (such as imports, callers, or configs), include them in 'dependent_patches' with exact search_block and ready_code_fix so they can be aut
```

3. Format Alternative Task Generation Prompt

Target File:	src/agent.py
Category:	improvement
Why / ROI:	Guarantees alternative suggestions can be auto-applied without syntax errors or missing search blocks.

Before (Original Code):

```
f"REJECTED FIX SNIPPET:\n{task.get('ready_code_fix')}\n\n"
"CRITICAL: You MUST solve the EXACT SAME core bug/issue described in REJECTED TASK TITLE, "
"but using a different code pattern, library, or architectural approach. Do NOT switch to a different bug."
)
if project_context:
prompt += f"\n\nPROJECT CONTEXT:\n{project_context[:40000]}"

response = self.client.models.generate_content(
model=self.model_name,
contents=prompt,
config=types.GenerateContentConfig(
system_instruction=self.skill_instructions,
response_mime_type="application/json",
response_schema=BacklogTask,
temperature=0.4,
),
)
```

After (Applied Fix):

```
f"REJECTED FIX SNIPPET:\n{task.get('ready_code_fix')}\n\n"
"CRITICAL: You MUST solve the EXACT SAME core bug/issue described in REJECTED TASK TITLE, "
"but using a different code pattern, library, or architectural approach. Do NOT switch to a different bug.\n"
"Provide EXACT verbatim search_block and complete, executable ready_code_fix without line numbers or ellipses."
)
if project_context:
prompt += f"\n\nPROJECT CONTEXT:\n{project_context[:40000]}"

response = self.client.models.generate_content(
model=self.model_name,
contents=prompt,
config=types.GenerateContentConfig(
system_instruction=self.skill_instructions,
response_mime_type="application/json",
response_schema=BacklogTask,
temperature=0.2,
),
)
```

4. Clean Snippets and Preserve Indentation

Target File:	src/main.py
Category:	improvement

Why / ROI:	Prevents auto-apply failures caused by indentation loss or accidental line-number prefixes in generated code.

Before (Original Code):

```
def apply_text_patch(content: str, search_block: str, replacement: str) -> str | None:
"""Pure function to safely apply search-and-replace with newline normalization."""
norm_content = content.replace("\r\n", "\n")
norm_search = search_block.replace("\r\n", "\n")
norm_replace = replacement.replace("\r\n", "\n")

target_match = norm_search if norm_search and norm_search in norm_content else (
norm_search.strip() if norm_search and norm_search.strip() in norm_content else None
)
if not target_match:
return None

actual_fix = norm_replace if target_match == norm_search else norm_replace.strip()
return norm_content.replace(target_match, actual_fix, 1)
```

After (Applied Fix):

```
def clean_patch_text(text: str) -> str:
"""Removes accidental line number prefixes (e.g. '12: ') emitted by LLMs while preserving indentation."""
import re
cleaned_lines = [
re.sub(r"^\s*\d+:\s?", "", line)
for line in text.replace("\r\n", "\n").split("\n")
]
return "\n".join(cleaned_lines)

def apply_text_patch(content: str, search_block: str, replacement: str) -> str | None:
"""Pure function to safely apply search-and-replace with newline and line-number normalization."""
norm_content = content.replace("\r\n", "\n")
norm_search = clean_patch_text(search_block).strip("\r\n")
norm_replace = clean_patch_text(replacement).strip("\r\n")

if norm_search and norm_search in norm_content:
return norm_content.replace(norm_search, norm_replace, 1)

# Fallback to trimmed search block if trailing blank lines differ
trimmed_search = norm_search.strip()
if trimmed_search and trimmed_search in norm_content:
return norm_content.replace(trimmed_search, norm_replace.strip(), 1)

return None
```

5. Preserve Code Indentation on Apply

Target File:	src/main.py
Category:	improvement
Why / ROI:	Directly eliminates indentation corruption when applying AI-generated code fixes.

Before (Original Code):

```
patches_to_apply = [{
"file_path": task["file_path"],
"search_block": (task.get("search_block") or "").strip(),
"ready_code_fix": task["ready_code_fix"].strip()
}]
```

After (Applied Fix):

```
patches_to_apply = [{
"file_path": task["file_path"],
"search_block": (task.get("search_block") or "").strip("\r\n"),
"ready_code_fix": (task.get("ready_code_fix") or "").strip("\r\n")
}]
```